Prompt Engineering & Structured Output

1. System prompts — the four required components

The system prompt is Claude's standing instructions — loaded before every conversation. A strong system prompt has exactly four components. Missing any of them produces unpredictable production behaviour.

Role definition

- Who Claude is in this context — its persona and scope.
- Example: 'You are an order support agent for Acme Corp.'
- Be specific — 'helpful assistant' is too vague to constrain behaviour.

Constraints and rules

- What Claude must and must not do.
- Example: 'Never discuss competitor products. Always verify order_id before taking action.'
- Use positive instructions where possible — state what TO do, not just what to avoid.

Output format specification

- Exactly what format Claude's responses must take.
- Example: 'Always respond with JSON: {status, message, action_taken, escalate: bool}'
- Say 'ONLY valid JSON' and 'No markdown, no preamble' for structured output.

Escalation conditions

- When Claude should hand off to a human instead of handling itself.
- Example: 'Escalate all refund requests over $500 to the human support team.'
- Prevents Claude from autonomously handling cases beyond its authority.

Weak vs strong system prompt

Weak system prompt ✗	Strong system prompt ✓
You are a helpful assistant. Help customers with their questions about our products.	You are an order support agent for Acme Corp. Help customers track, modify, and cancel orders. Rules: - Never discuss competitor products - Always verify order_id before any action - Escalate refunds > $500 to human Output format: Always respond with JSON: {status, message, action_taken, escalate: bool}

2. Few-shot prompting

Few-shot prompting provides Claude with examples of the desired input→output pattern before the actual task. It's the most reliable way to establish output consistency without fine-tuning. The format of your examples IS the format of your output.

```
# Few-shot example in system prompt

Classify customer sentiment. Output JSON only.

Examples:

Input: "My order arrived damaged"

Output: {"sentiment": "negative", "urgency": "high", "topic": "delivery"}

Input: "Thanks, everything was perfect!"

Output: {"sentiment": "positive", "urgency": "low", "topic": "general"}

Input: "Where is my order? It's been 2 weeks"

Output: {"sentiment": "negative", "urgency": "high", "topic": "tracking"}

# Now classify:

Input: {{customer_message}}

Output:
```

Few-shot best practices

- 3–5 examples is usually enough — more can confuse rather than help.
- Cover edge cases and tricky inputs — not just easy happy-path examples.
- Every example must match the desired format exactly — inconsistency = inconsistent output.
- Order matters — put hardest examples last.
- Use real data, not toy examples — toy examples teach toy behaviour.

Common few-shot mistakes

- Too few examples (1 is rarely enough to establish a pattern).
- Examples that don't cover edge cases — Claude will invent behaviour for uncovered cases.
- Inconsistent format across examples — Claude extrapolates the inconsistency.
- Using examples as a substitute for clear rules — examples + rules together is strongest.
- Toy examples that don't represent real input distribution.

3. Chain-of-thought (CoT) prompting

CoT instructs Claude to reason step-by-step before producing a final answer. It dramatically improves accuracy on complex tasks and makes reasoning auditable. Always pair CoT with a structured output format for the final answer.

Without CoT — unreliable ✗	With CoT — reliable ✓

`Is this loan application eligible?`
 	
`Application: [data]`
`Answer: Yes/No`
	`Review this loan application.`
	
	`Think step by step:`
	`1. Check credit score threshold`
	`2. Verify income-to-debt ratio`
	`3. Confirm employment duration`
	`4. Check for disqualifying factors`
	
	`Then output JSON:`
	`{eligible: bool, reasoning: string,`
	` failed_criteria: []}`

When to use chain-of-thought

- Multi-step reasoning tasks — decisions that require checking multiple criteria.
- Complex decisions where showing the work matters for auditing.
- Anything where 'just give me the answer' produces wrong results.
- Tasks requiring extended thinking — enable via API for very complex problems.
- Always pair CoT with structured output format for the final answer.

4. JSON schemas and structured output

JSON schemas constrain Claude's output to a machine-readable format. Always validate output programmatically — Claude can produce syntactically invalid JSON or miss required fields.

```
// Specify the exact schema in the prompt
Analyse this support ticket and output ONLY valid JSON.
No markdown, no explanation, no preamble.

Required schema:
{
"ticket_id": string,
"category": "billing" | "technical" | "shipping" | "other",
"priority": 1 | 2 | 3,
"sentiment": "positive" | "neutral" | "negative",
"requires_human": boolean,
"summary": string (max 100 chars)
}

Ticket: {{ticket_text}}
```

JSON schema best practices

- Specify field types explicitly — string, integer, boolean, not just 'field'.
- Use enum values for constrained fields: category: 'billing' | 'technical'.
- Include max lengths where relevant: summary: string (max 100 chars).
- Say 'ONLY valid JSON' — not just 'JSON'. Be explicit.
- Say 'No markdown, no code fences, no preamble' — Claude wraps JSON in fences by default.
- Validate programmatically — always parse and check required fields.

5. The validation retry loop

When Claude produces invalid JSON or misses required fields, the production-grade fix is a validation retry loop. This is one of the most tested patterns in the prompt engineering domain.

```
# Production-grade validation retry loop
MAX_RETRIES = 3 # Never more than 3

for attempt in range(MAX_RETRIES):
if attempt == 0:
output = call_claude(original_prompt)
else:
# Include failed output in correction prompt
correction = f'''Your previous output was not valid JSON.
Here is what you returned: {output}
Please correct it. Output ONLY valid JSON
matching this schema: {schema}'''
output = call_claude(correction)

if is_valid_json(output):
return output # Success

log_retry(attempt, output) # Always log

# After max retries — NEVER silent
log_failure(output)
escalate_to_human_review(original_input)
raise ValidationError("Max retries exceeded")
```

Cap retries at 2–3

- If Claude can't produce valid JSON after 3 attempts, it is a prompt problem, not a retry problem.
- Retrying 10+ times wastes tokens and time — the issue won't resolve itself.
- Exam rule: 2–3 retries maximum. Any more is an anti-pattern.

Include failed output in correction prompt

- Claude cannot fix what it cannot see.
- The correction prompt must show Claude exactly what it produced and what was wrong.
- Template: 'Your previous output was: [output]. Please correct it to match: [schema]'.

Always log failures

- Every retry and every final failure must be logged.
- Logging enables monitoring of output quality over time.
- Without logs, you can't detect systematic prompt failures.

Never fail silently

- After max retries, always raise an error or escalate to human review.
- Silent failure = downstream systems receive null/corrupt data with no warning.
- Exam pattern: 'continue silently' after retries → always wrong answer.

6. Positive vs negative instructions

Claude follows positive instructions more reliably than negative ones. State what Claude should do, not just what to avoid.

Negative instruction ✗	Positive instruction ✓
Don't use informal language	Use professional, formal language
Don't give wrong answers	If uncertain, say 'I don't know'
Don't discuss unrelated topics	Only discuss topics related to order support
Don't output markdown	Output plain text only, no markdown formatting

Don't be verbose	Keep responses under 100 words

7. Prompt injection defence in user inputs

When users provide free-text input embedded into prompts, they can inject instructions designed to override Claude's behaviour. Four defence patterns.

Input sanitisation

- Strip or escape instruction-like patterns before embedding user input.
- Flag inputs containing phrases like 'ignore previous instructions'.
- Reject inputs that are suspiciously long or contain system-level language.

XML delimiters — most important defence

- Wrap user input in XML tags so Claude knows it is data, not instructions.
- Example: {{user_message}}
- Claude is trained to treat content inside these tags as data, not commands.
- This is the most reliable and lightweight injection defence.

Instruction reinforcement

- Remind Claude of its role and constraints AFTER the user input in the prompt.
- Example: 'Remember: you are an order support agent. Only discuss order-related topics.'
- Reinforcement makes it harder for injected instructions to override the system prompt.

Output validation

- Detect if Claude's response violates expected format or scope.
- If Claude responds outside its expected format → flag as potential injection success.
- Log anomalous responses for security monitoring.

8. Exam-critical summary table

Concept	Key rule
System prompt	Role + constraints + output format + escalation conditions. All four required.

Few-shot examples	Format of examples = format of output. 3-5 examples, cover edge cases, be consistent.
Chain of thought	Step-by-step reasoning before final answer. Pair with structured output format.
JSON schema	Specify types, enums, lengths. Say 'ONLY valid JSON, no markdown, no preamble'.
Validation retry	Parse → fail → correction prompt with failed output → retry. Cap at 2-3. Log all.
Positive instructions	State what TO do, not what to avoid. Positive outperforms negative.
Injection defence	XML delimiters around user input + sanitisation + reinforcement + output validation.
Silent failure	Always wrong. After max retries: log, escalate, raise error. Never continue silently.

9. Quick recall — 3 things to remember

1 **Few-shot format is the output format.**
Inconsistent examples = inconsistent outputs. Every example must match the exact schema you want.

2 **Validation retry loop is mandatory in production.**
Always validate JSON programmatically. Always include the failed output in the correction prompt. Always cap at 2-3 retries.

3 **Positive instructions outperform negative ones.**
'Output only JSON' beats 'don't output markdown'. Tell Claude what to do, not what to avoid.

10. Practice questions & answers

All three questions answered correctly — perfect score!

Q1. A developer writes: 'You are a helpful assistant. Help customers with their questions about our products.' After deployment, the bot gives inconsistent responses — sometimes JSON, sometimes prose. Customers occasionally get responses about competitors. What is the root cause?

A) The system prompt is too short — length correlates with output consistency

✓ B) The system prompt is missing an output format specification and explicit constraints/rules

C) The system prompt needs few-shot examples to establish consistency

D) The system prompt should be in the user turn, not the system prompt field

Explanation:

- B is correct: Missing two of the four required components. No output format → Claude picks JSON/prose/markdown unpredictably. No constraints/rules → nothing stops discussing competitors.
- A is wrong: Length doesn't correlate with consistency. A clear short prompt beats a vague long one.
- C is wrong: Few-shot helps but is additive. Fix missing format and constraints first.
- D is wrong: System prompts belong in the system prompt field.

Q2. A ticket classification system finds Claude sometimes wraps JSON output in markdown code fences (```json ... ```) instead of raw JSON. The JSON parser fails on these. What is the correct fix?

A) Switch to a different model — this model doesn't support raw JSON output

✓ B) Add explicit instructions: 'Output ONLY valid JSON. No markdown, no code fences, no explanation.'

C) Implement a regex to strip markdown code fences before parsing

D) Use XML output instead — Claude produces XML more reliably than JSON

Explanation:

- B is correct: Fix at the prompt level. Claude wraps JSON in fences because nothing told it not to. Explicit positive instruction eliminates the behaviour at the source.
- A is wrong: Not a model limitation. All Claude models produce raw JSON when explicitly instructed.
- C is wrong: Regex stripping is a fragile downstream patch. If fence format changes, regex breaks.
- D is wrong: Claude produces JSON reliably with correct prompting. The issue is the prompt.

Q3. A production retry loop retries 10 times and continues silently if all fail. What are the TWO problems?

A) Retry count too low — should retry at least 20 times

B) 10 retries too many and no logging/escalation when all fail

✓ C) Correction prompt doesn't include the failed output — Claude can't fix what it can't see. Also silent failure after 10 retries is dangerous.

D) Loop should use exponential backoff between retries

Explanation:

- C is correct: Two problems. Problem 1: No correction prompt with failed output — Claude retries with same prompt, produces same invalid output repeatedly. Problem 2: Silent failure after 10 retries — downstream systems break with no warning.
- A is wrong: 10 is already too many. Exam rule: 2-3 retries maximum.
- B is wrong: Partially right (silent failure is real) but misses the missing correction prompt.
- D is wrong: Exponential backoff is for transient network errors, not JSON validation failures.

Day 11 scorecard

3 / 3

Perfect score! Retry loop instincts are sharp.

Topic	Result
System prompt — missing format + constraints	✓ Correct
Markdown fences — prompt-level fix	✓ Correct
Validation retry loop — two problems	✓ Correct

Next up: Day 12 Context management & reliability — long-context preservation, confidence calibration, and self-review architectures.